

REVERSE YOUR FUTURE

You can't secure what you can't control: opening up the RH850

www.quarkslab.com

Secure our Streets 2026

Philippe AZALBERT
Damien CAUQUIL
Julien RAKOTOMALALA

Quarkslab
REVERSE YOUR FUTURE

Speakers presentation



Damien CAUQUIL
Virtualabs

- R&D engineer
- Hack all the things
- Reverse weird stuff
- 2.4Ghz wizard



Julien RAKOTOMALALA

- R&D engineer
- Track racer
- Hack wheeled vehicles
- Truck/car in a box guru

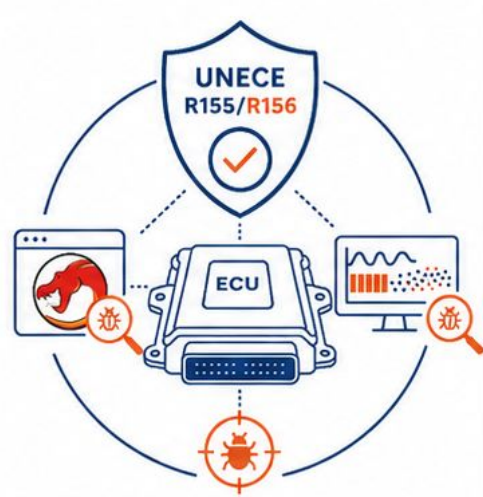


Philippe AZALBERT

- R&D engineer
- Car hacking trainer
- Hardware glitcher

Context: ECU firmware analysis

- As a cybersecurity company, we are tasked to assess ECUs at various development stage to check critical parts:
 - **Firmware update** process
 - **Secure boot** verification
 - CAN/Ethernet **message authenticity**
 - **External communication** analysis
 - ...
- Assessment are mostly performed in a **grey-box** approach, from time to time, in a **black-box** one



Hardware/software constraints

- For most audit, only a **raw firmware** is provided with the ECU
- **Dynamic analysis** capability is possible **using the same environment** when having source code or compiled code with symbols
- Those tools being proprietary, **no or limited customization** is permitted
- They are often tied to the **Windows operating system**

Automotive microcontroller & open-source support

Manufacturer	Infineon/Aurix	STMicroelectronics	Renesas
Microcontroller	Tricore	SPC5x	RH850
Architecture	Tricore	Power PC	V850x
Mission %	20%	10%	70%
Emulation (QEMU)	Yes	Yes	No
GDB support	Lack specific hardware interface support		

The quest for the missing instruction set

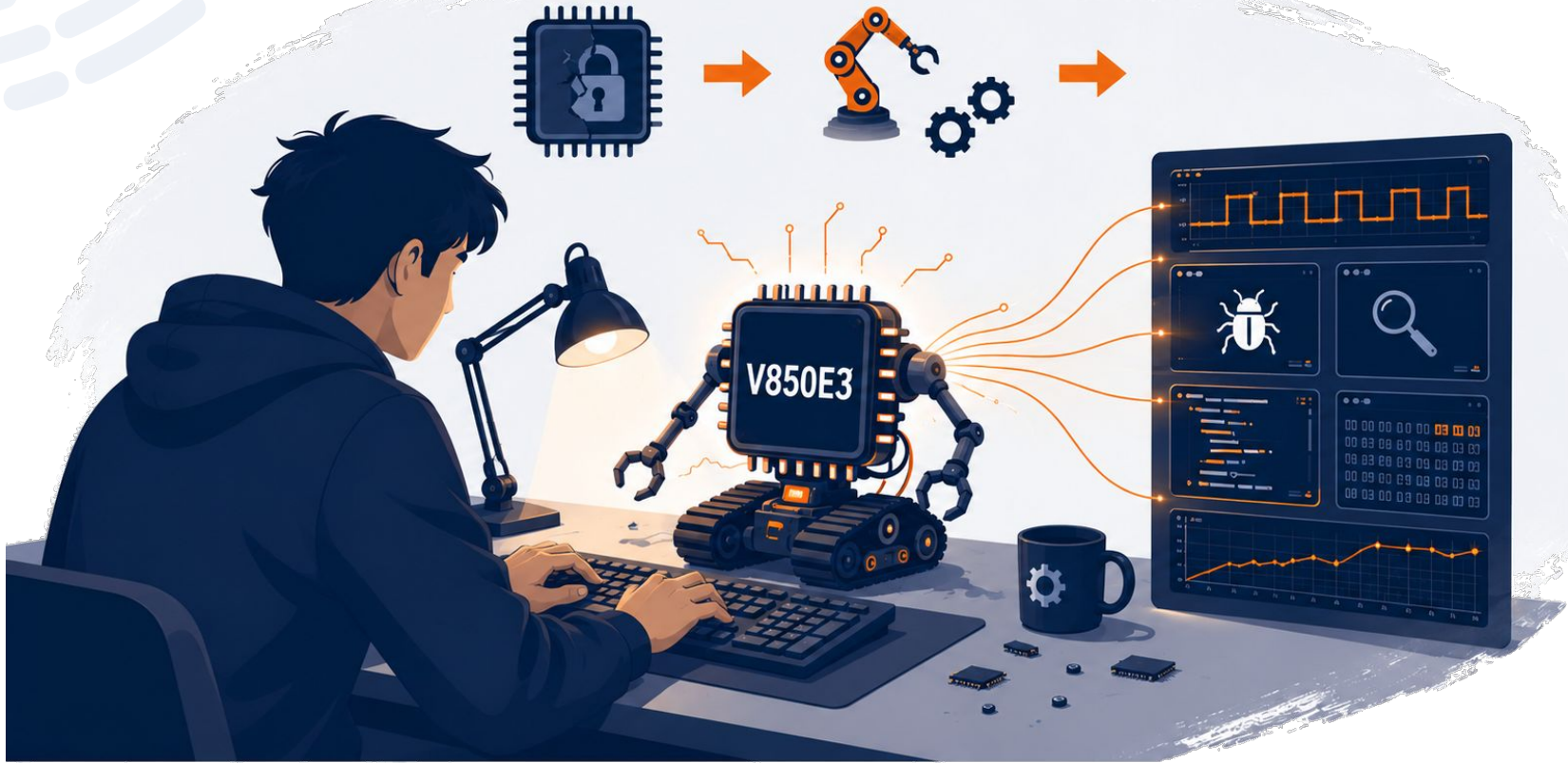
- We mainly assess the following RH850 variants:

Microcontroller	RH850/F1x	RH850/E1x	RH850/U2A
Core	G3K/G3KH	G3M/G3MH	G4MH
Instruction set	V850E3		

- Only **static analysis** can be performed on these microcontrollers, lacking:
 - Emulation**
 - Fuzzing**
 - Debugging**

Can we overcome these limitations?

Challenge accepted!





Building an RH850 emulator

Developing an Unicorn based V850E3 emulator

- Unicorn Engine is based on **QEMU**
- It provides several **bindings**:
 - Python
 - Rust
 - Go
 - ...
- It supports **instrumentation**
- **Hooks** allow to control execution flow



V850E3 support for QEMU

- A fork by Marko Klopčič from iSYSTEM Labs exists¹, but lacks for:
 - Exceptions
 - FPU instructions
- Let's improve it!

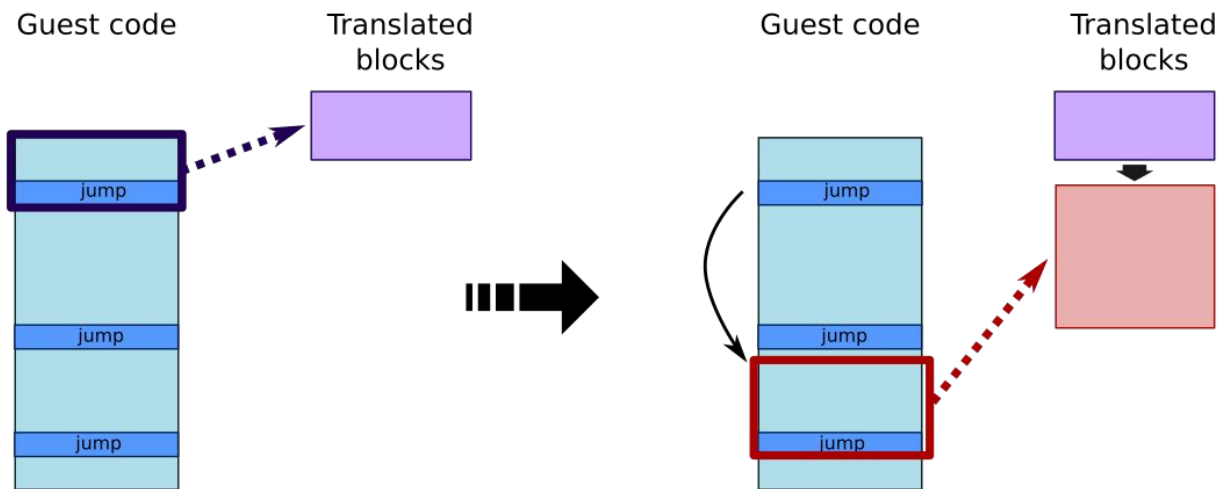


1: <https://github.com/markok314/qemu>

QEMU main concepts

- Instructions are **translated**, not interpreted directly, being more efficient than direct emulation
- Translation is handled by the **Tiny Code Generator (TCG)**
- Each emulated instruction generates **Intermediate Representation (IR)** before being translated into **native host code**
- Native code is **cached** and **linked** into **optimized execution blocks** (Translation Blocks)
- TCG exposes an **API** of basic functions, letting each CPU implementation generate its own IR code

TCG guest code translation



IR generator example

ADD									
	Add								
Instruction format	(1) ADD reg1, reg2 (2) ADD imm5, reg2								
Operation	(1) $GR[reg2] \leftarrow GR[reg2] + GR[reg1]$ (2) $GR[reg2] \leftarrow GR[reg2] + \text{sign-extend}(imm5)$								
Format	(1) Format I (2) Format II								
Op code	(1) <table><tr><td>15</td><td>0</td></tr><tr><td>rrrrr001110RRRRR</td><td></td></tr></table> (2) <table><tr><td>15</td><td>0</td></tr><tr><td>rrrrr010010iiii</td><td></td></tr></table>	15	0	rrrrr001110RRRRR		15	0	rrrrr010010iiii	
15	0								
rrrrr001110RRRRR									
15	0								
rrrrr010010iiii									
Flag	CY 1 if a carry occurs from MSB; otherwise, 0. OV 1 if an overflow occurs; otherwise, 0. S 1 if the result of an operation is negative; otherwise, 0. Z 1 if the result of an operation is 0; otherwise 0. SAT -								
Instruction	(1) ADD Add Register (2) ADD Add Immediate (5-bit)								

IR generator example

```
static void gen_intermediate_add_reg_reg (DisasContext *ctx, int rs1, int rs2)
{
    /* Retrieve the TCG context from Unicorn's disassembly context. */
    TCGContext *tcg_ctx = ctx->uc->tcg_ctx;

    /* Create two temporary TCG variables. */
    TCGv r1 = tcg_temp_new(tcg_ctx);
    TCGv r2 = tcg_temp_new(tcg_ctx);
    tcg_gen_mov_tl(tcg_ctx, r1, cpu_gpr[rs1]); /* Copy register value into temp TCG variable */
    tcg_gen_mov_tl(tcg_ctx, r2, cpu_gpr[rs2]);

    /* Add r1 and r2 and write the result into tcg_result */
    TCGv r2 = tcg_temp_new(tcg_result);
    tcg_gen_add_tl(tcg_ctx, tcg_result, r2, r1);

    /* Write the result into general-purpose register designed by index rs2 */
    tcg_gen_mov_tl(tcg_ctx, cpu_gpr[rs2], tcg_result);

    /* Update flags */
    gen_flags_on_add(tcg_ctx, r1, r2);
}
```

IR generator example

```
static void gen_intermediate_add_reg_reg (DisasContext *ctx, int rs1, int rs2)
{
    /* Retrieve the TCG context from Unicorn's disassembly context. */
    TCGContext *tcg_ctx = ctx->uc->tcg_ctx;

    /* Create two temporary TCG variables. */
    TCGv r1 = tcg_temp_new(tcg_ctx);
    TCGv r2 = tcg_temp_new(tcg_ctx);
    tcg_gen_mov_tl(tcg_ctx, r1, cpu_gpr[rs1]); /* Copy register value into temp TCG variable */
    tcg_gen_mov_tl(tcg_ctx, r2, cpu_gpr[rs2]);

    /* Add r1 and r2 and write the result into tcg_result */
    TCGv r2 = tcg_temp_new(tcg_result);
    tcg_gen_add_tl(tcg_ctx, tcg_result, r2, r1);

    /* Write the result into general-purpose register designed by index rs2 */
    tcg_gen_mov_tl(tcg_ctx, cpu_gpr[rs2], tcg_result);

    /* Update flags */
    gen_flags_on_add(tcg_ctx, r1, r2);
}
```

IR generator example

```
static void gen_intermediate_add_reg_reg (DisasContext *ctx, int rs1, int rs2)
{
    /* Retrieve the TCG context from Unicorn's disassembly context. */
    TCGContext *tcg_ctx = ctx->uc->tcg_ctx;

    /* Create two temporary TCG variables. */
    TCGv r1 = tcg_temp_new(tcg_ctx);
    TCGv r2 = tcg_temp_new(tcg_ctx);
    tcg_gen_mov_tl(tcg_ctx, r1, cpu_gpr[rs1]); /* Copy register value into temp TCG variable */
    tcg_gen_mov_tl(tcg_ctx, r2, cpu_gpr[rs2]);

    /* Add r1 and r2 and write the result into tcg_result */
    TCGv r2 = tcg_temp_new(tcg_result);
    tcg_gen_add_tl(tcg_ctx, tcg_result, r2, r1);

    /* Write the result into general-purpose register designed by index rs2 */
    tcg_gen_mov_tl(tcg_ctx, cpu_gpr[rs2], tcg_result);

    /* Update flags */
    gen_flags_on_add(tcg_ctx, r1, r2);
}
```

IR generator example

```
static void gen_intermediate_add_reg_reg (DisasContext *ctx, int rs1, int rs2)
{
    /* Retrieve the TCG context from Unicorn's disassembly context. */
    TCGContext *tcg_ctx = ctx->uc->tcg_ctx;

    /* Create two temporary TCG variables. */
    TCGv r1 = tcg_temp_new(tcg_ctx);
    TCGv r2 = tcg_temp_new(tcg_ctx);
    tcg_gen_mov_tl(tcg_ctx, r1, cpu_gpr[rs1]); /* Copy register value into temp TCG variable */
    tcg_gen_mov_tl(tcg_ctx, r2, cpu_gpr[rs2]);

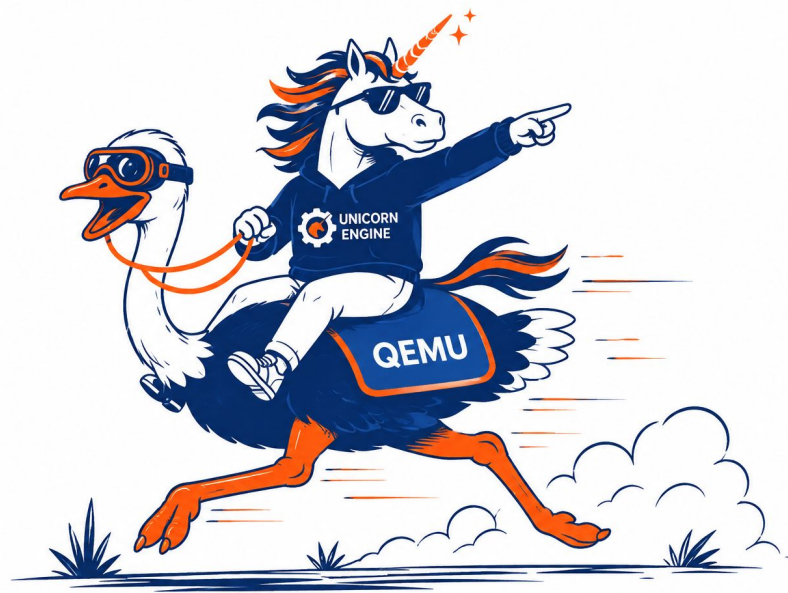
    /* Add r1 and r2 and write the result into tcg_result */
    TCGv r2 = tcg_temp_new(tcg_result);
    tcg_gen_add_tl(tcg_ctx, tcg_result, r2, r1);

    /* Write the result into general-purpose register designed by index rs2 */
    tcg_gen_mov_tl(tcg_ctx, cpu_gpr[rs2], tcg_result);

    /* Update flags */
    gen_flags_on_add(tcg_ctx, r1, r2);
}
```

From QEMU to Unicorn Engine

- Next step: implement V850 QEMU CPU into Unicorn Engine:
 - CPU declaration
 - Set translators
 - Define registers
 - ...
- Full write-up: [Quarkslab's blog](https://blog.quarkslab.com/emulating-rh850-architecture-with-unicorn-engine.html)¹



1: <https://blog.quarkslab.com/emulating-rh850-architecture-with-unicorn-engine.html>

Unicorn emulation example

```
from unicorn import *
from unicorn.rh850_const import *

UDS_PAYLOAD = b'\x22\xF2\xAA' # Read DID 0xF2AA

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

with open("./firmware.bin", "rb") as f: # Loading firmware
    app = f.read()
uc.mem_map(0x0000C000, 0x00008000)
uc.mem_write(0x0000C000, app)

uc.mem_map(0x60000000, 0x00010000) # Stack initialization
uc.reg_write(UC_RH850_REG_SP, 0x60000000 + 0x00010000)
uc.mem_map(0xFE000000, 0x02000000) # RAM initialization
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc.mem_write(0xFFFF0625, b'\x03') # UDS message length
uc.mem_write(0xFEDD93CD, b'\x22\xF2\xAA') # UDS message payload

uc.emu_start(0x00018DAE, 0x00018EAE, timeout=0, count=0)
uc.emu_stop()
```

Unicorn emulation example

```
from unicorn import *
from unicorn.rh850_const import *

UDS_PAYLOAD = b'\x22\xF2\xAA' # Read DID 0xF2AA

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

with open("./firmware.bin", "rb") as f: # Loading firmware
    app = f.read()
uc.mem_map(0x0000C000, 0x00008000)
uc.mem_write(0x0000C000, app)

uc.mem_map(0x60000000, 0x00010000) # Stack initialization
uc.reg_write(UC_RH850_REG_SP, 0x60000000 + 0x00010000)
uc.mem_map(0xFE000000, 0x02000000) # RAM initialization
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc.mem_write(0xFFFF0625, b'\x03') # UDS message length
uc.mem_write(0xFEDD93CD, b'\x22\xF2\xAA') # UDS message payload

uc.emu_start(0x00018DAE, 0x00018EAE, timeout=0, count=0)
uc.emu_stop()
```

Unicorn emulation example

```
from unicorn import *
from unicorn.rh850_const import *

UDS_PAYLOAD = b'\x22\xF2\xAA' # Read DID 0xF2AA

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

with open("./firmware.bin", "rb") as f: # Loading firmware
    app = f.read()
    uc.mem_map(0x0000C000, 0x00008000)
    uc.mem_write(0x0000C000, app)

uc.mem_map(0x60000000, 0x00010000) # Stack initialization
uc.reg_write(UC_RH850_REG_SP, 0x60000000 + 0x00010000)
uc.mem_map(0xFE000000, 0x02000000) # RAM initialization
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc.mem_write(0xFFFF0625, b'\x03') # UDS message length
uc.mem_write(0xFEDD93CD, b'\x22\xF2\xAA') # UDS message payload

uc.emu_start(0x00018DAE, 0x00018EAE, timeout=0, count=0)
uc.emu_stop()
```

Unicorn emulation example

```
from unicorn import *
from unicorn.rh850_const import *

UDS_PAYLOAD = b'\x22\xF2\xAA' # Read DID 0xF2AA

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

with open("./firmware.bin", "rb") as f: # Loading firmware
    app = f.read()
uc.mem_map(0x0000C000, 0x00008000)
uc.mem_write(0x0000C000, app)

uc.mem_map(0x60000000, 0x00010000) # Stack initialization
uc.reg_write(UC_RH850_REG_SP, 0x60000000 + 0x00010000)
uc.mem_map(0xFE000000, 0x02000000) # RAM initialization
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc.mem_write(0xFFFF0625, b'\x03') # UDS message length
uc.mem_write(0xFEDD93CD, b'\x22\xF2\xAA') # UDS message payload

uc.emu_start(0x00018DAE, 0x00018EAE, timeout=0, count=0)
uc.emu_stop()
```

Unicorn emulation example

```
from unicorn import *
from unicorn.rh850_const import *

UDS_PAYLOAD = b'\x22\xF2\xAA' # Read DID 0xF2AA

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

with open("./firmware.bin", "rb") as f: # Loading firmware
    app = f.read()
uc.mem_map(0x0000C000, 0x00008000)
uc.mem_write(0x0000C000, app)

uc.mem_map(0x60000000, 0x00010000) # Stack initialization
uc.reg_write(UC_RH850_REG_SP, 0x60000000 + 0x00010000)
uc.mem_map(0xFE000000, 0x02000000) # RAM initialization
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc.mem_write(0xFFFF0625, b'\x03') # UDS message length
uc.mem_write(0xFEDD93CD, b'\x22\xF2\xAA') # UDS message payload

uc.emu_start(0x00018DAE, 0x00018EAE, timeout=0, count=0)
uc.emu_stop()
```

Unicorn emulation example

```
from unicorn import *
from unicorn.rh850_const import *

UDS_PAYLOAD = b'\x22\xF2\xAA' # Read DID 0xF2AA

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

with open("./firmware.bin", "rb") as f: # Loading firmware
    app = f.read()
uc.mem_map(0x0000C000, 0x00008000)
uc.mem_write(0x0000C000, app)

uc.mem_map(0x60000000, 0x00010000) # Stack initialization
uc.reg_write(UC_RH850_REG_SP, 0x60000000 + 0x00010000)
uc.mem_map(0xFE000000, 0x02000000) # RAM initialization
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc.mem_write(0xFFFF0625, b'\x03') # UDS message length
uc.mem_write(0xFEDD93CD, b'\x22\xF2\xAA') # UDS message payload

uc.emu_start(0x00018DAE, 0x00018EAE, timeout=0, count=0)
uc.emu_stop()
```

Unicorn emulation result

```
user@qb:~/RH850_emulation$ ./emulator_harness.py
```

```
[INFO] UDS payload: 22F2AA
```

```
[INFO] Emulating function RDBI
```

```
[INFO] Starting emulation @0x00018DAE to 0x00018EAE
```

```
[INFO] Execution ended
```

```
[INFO] 00000000 00 62 F2 AA FF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ·bðª| Ÿ··| ···| ···|
```

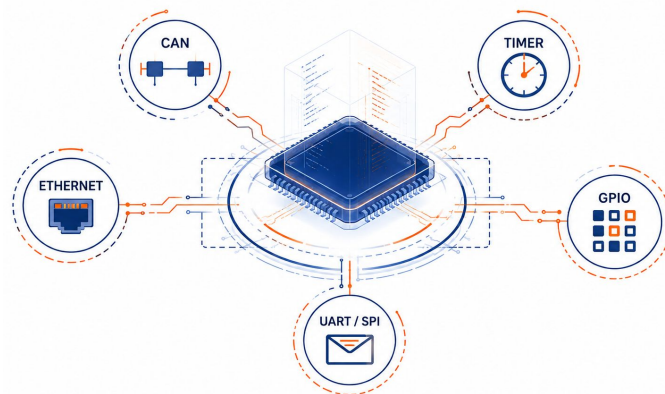
Roadblock: peripheral registers

3BC.2.2.3 Registers for Buffer Control

(1) List of Buffer Control Registers

Table 3BC.57 Buffer Control Register (Base Address: FFC5 B000_H)

Module Name	Address Offset	Size (Byte)	Register Name	Abbreviation	Right	R/W	Operable Bit				Value after Reset
							1	8	16	32	
FBUF_CTRL	+000 _H	4	Flash buffer clear control register	FBUFCTL	—	R/W	—	✓	✓	✓	0000 0000 _H



Unicorn hooks

- `UC_HOOK_INTR`: hook all interrupt/syscall events
 - `UC_HOOK_INSN`: hook a particular instruction (not all instructions supported)
 - `UC_HOOK_MEM_READ`: hook memory read events
 - `UC_HOOK_MEM_WRITE`: hook memory write events
 - `UC_HOOK_MEM_READ_UNMAPPED`: hook for memory read on unmapped memory
 - `UC_HOOK_MEM_WRITE_UNMAPPED`: hook for invalid memory write events
 - `UC_HOOK_MEM_FETCH_UNMAPPED`: hook for invalid memory fetch for execution events
-
- `UC_HOOK_CODE`: hook a range of code
 - `UC_HOOK_BLOCK`: hook basic blocks
 - `UC_HOOK_MEM_READ_PROT`: hook for memory read on read-protected memory
 - `UC_HOOK_MEM_WRITE_PROT`: hook for memory write on write-protected memory
 - `UC_HOOK_MEM_FETCH_PROT`: hook for memory fetch on non-executable memory
 - `UC_HOOK_MEM_READ_AFTER`: hook memory read events, but only successful access

Fuzzing with AFL++ in Unicorn mode



```
american fuzzy lop ++4.20c {default} (./harness) [explore]
process timing                                     overall results
  run time : 0 days, 0 hrs, 0 min, 1 sec          cycles done : 0
  last new find : 0 days, 0 hrs, 0 min, 0 sec      corpus count : 24
  last saved crash : 0 days, 0 hrs, 0 min, 0 sec   saved crashes : 13
  last saved hang : none seen yet                  saved hangs : 0
cycle progress                                     map coverage
  now processing : 0.1 (0.0%)                     map density : 1.42% / 1.57%
  runs timed out : 0 (0.00%)                     count coverage : 1.79 bits/tuple
stage progress                                     findings in depth
  now trying : havoc                             favored items : 1 (4.17%)
  stage execs : 138/3.15M (0.00%)                 new edges on : 12 (50.00%)
  total execs : 330                             total crashes : 89 (13 saved)
  exec speed : 342.9/sec                         total tmouts : 13 (0 saved)
fuzzing strategy yields                           item geometry
  bit flips : 0/0, 0/0, 0/0                      levels : 2
  byte flips : 0/0, 0/0, 0/0                     pending : 24
  arithmetics : 0/0, 0/0, 0/0                    pend fav : 1
  known ints : 0/0, 0/0, 0/0                     own finds : 23
  dictionary : 0/0, 0/0, 0/0, 0/0, 0/0           imported : 0
  havoc/splice : 0/0, 0/0                        stability : 100.00%
  py/custom/rq : unused, unused, unused, unused
  trim/eff : 0.00%/3, n/a
strategy: explore                                state: started :-)^C
[cpu000: 37%]

+++ Testing aborted by user +++
[+] We're done here. Have a nice day!
```

Unicorn harness example

```
from unicorn import *
from unicorn.rh850_const import *
from unicornafl import uc_afl_fuzz

def place_input_cb(uc: Uc, content: bytes, persistent_round, data):
    uc.mem_write(STACK_ADDRESS+100, content)

    if content[0] & 0x51 == 0x51:
        uc.reg_write(arm_const.UC_ARM_REG_R0, 0)
    else:
        uc.reg_write(arm_const.UC_ARM_REG_R0, 1)
    uc.reg_write(arm_const.UC_ARM_REG_R1, STACK_ADDRESS+100)

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

# Load firmware and map memory sections
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE)          # Registers initialization

uc_afl_fuzz(uc=uc, input_file="@@", place_input_callback=place_input_cb, exits=[0x00018EAE],)
```

Unicorn harness example

```
from unicorn import *
from unicorn.rh850_const import *
from unicornafl import uc_afl_fuzz

def place_input_cb(uc: Uc, content: bytes, persistent_round, data):
    uc.mem_write(STACK_ADDRESS+100, content)

    if content[0] & 0x51 == 0x51:
        uc.reg_write(arm_const.UC_ARM_REG_R0, 0)
    else:
        uc.reg_write(arm_const.UC_ARM_REG_R0, 1)
    uc.reg_write(arm_const.UC_ARM_REG_R1, STACK_ADDRESS+100)

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

# Load firmware and map memory sections
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE)          # Registers initialization

uc_afl_fuzz(uc=uc, input_file="@@", place_input_callback=place_input_cb, exits=[0x00018EAE],)
```

Unicorn harness example

```
from unicorn import *
from unicorn.rh850_const import *
from unicornafl import uc_afl_fuzz

def place_input_cb(uc: Uc, content: bytes, persistent_round, data):
    uc.mem_write(STACK_ADDRESS+100, content)

    if content[0] & 0x51 == 0x51:
        uc.reg_write(arm_const.UC_ARM_REG_R0, 0)
    else:
        uc.reg_write(arm_const.UC_ARM_REG_R0, 1)
    uc.reg_write(arm_const.UC_ARM_REG_R1, STACK_ADDRESS+100)

uc = Uc(UC_ARCH_RH850, UC_MODE_LITTLE_ENDIAN)

# Load firmware and map memory sections
uc.reg_write(UC_RH850_REG_PC, 0x00018DAE) # Registers initialization

uc_afl_fuzz(uc=uc, input_file="@@", place_input_callback=place_input_cb, exits=[0x00018EAE],)
```





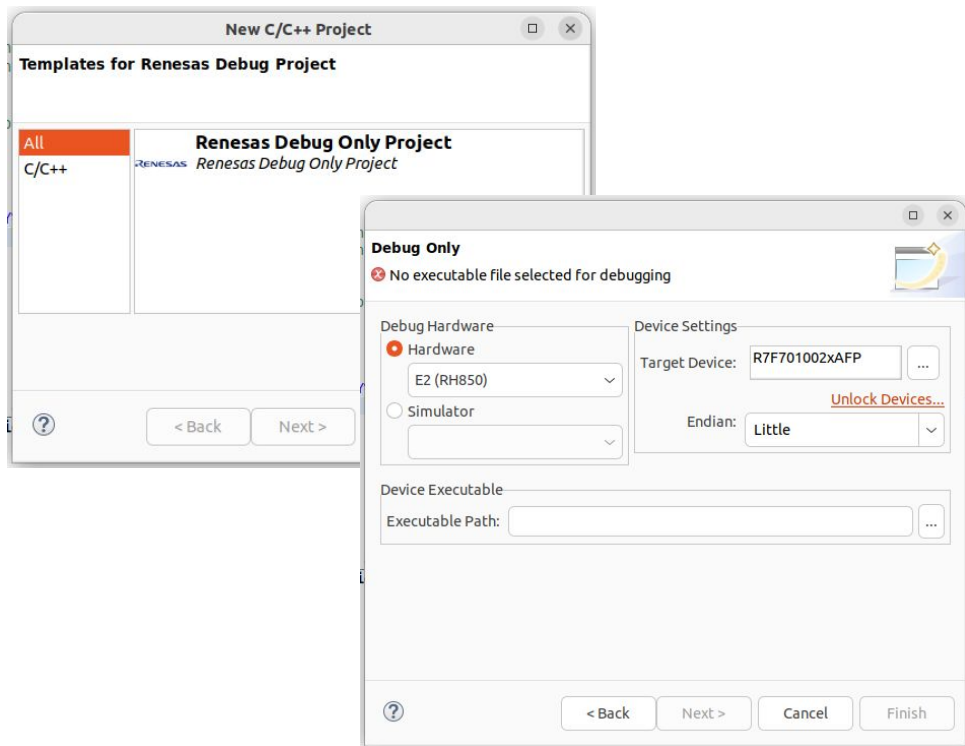
Debugging RH850

E2 Emulator device

- To read and write Flash memory of the RH850 we use the **E2 Emulator probe**
- It can be used for debugging using Renesas **e² studio**



- Renesas IDE based on **Eclipse**
- **Linux** support since January 2023
- Embedded **custom GDB tools**
 - e2-server-gdb
 - v850-elf-gdb
- Provides a debug only option
 - Requires an **ELF** file
 - Tied to Eclipse debug interface



Limitations

- Firmware we have to assess during security audit are retrieved:
 - With a raw dump of the RH850 microcontroller with **rfp-cli**
 - Or provided in **ODX/PDX update files**
- Embedded e² studio debugger lacks **automation** as:
 - **e2-server-gdb** is not supported by standard **gdb**
 - **e2-server-gdb** and **v850-elf-gdb** require specific parameters to be launched
- Can we have a solution for **agentic debugging** capability for example

ELF conversion

```
user@qb:~/RH850_debug$ rfp-cli -d RH850 -t e2 -vo 5 -r firmware.hex -osc 16 -format hex
```

Renesas Flash Programmer CLI V1.03

Module Version: V3.09.01.000

Connecting the tool (E2 emulator)

Interface: 2 wire UART

Connected to R7F701649

Reading data from the device

[Code Flash 1] 00000000 - 003FFFFFFF

Save file: ./firmware.hex

```
user@qb:~/RH850_debug$ objcopy -I ihex firmware.hex -O binary firmware.bin
```

```
user@qb:~/RH850_debug$ objcopy -I binary -O elf32-little --rename-section .data=.text  
--change-section-address .data=0x00000000 firmware.bin firmware.elf
```

Debugging with e² studio

The screenshot displays the e2 studio IDE interface. The top-left pane shows the 'Debug' view with 'Debug DebugOnly.launch' selected. The top-right pane shows the 'Disassembly' view for address 0x27628, displaying assembly instructions like 'br 0x27626', 'mov r0, r1', etc. The bottom-left pane shows the 'Debug console' with a tree view of the debug session, including 'Process [1] [cores: 0]' and 'Thread #1 1 (single core) [core: 0]'. The bottom-right pane shows the 'Console' output, which includes the GDB server startup message, raw options, and version information.

Break at address "0x0" with no debug information available, or outside of program code.

View Disassembly...

Configure when this editor is shown Preferences...

0x27628

00027624: br 0x27626
00027626: br 0x27626
00027628: mov r0, r1
0002762a: mov r0, r2
0002762c: mov r0, sp
0002762e: mov r0, gp
00027630: mov r0, r5
00027632: mov r0, r6
00027634: mov r0, r7
00027636: mov r0, r8
00027638: mov r0, r9
0002763a: mov r0, r10
0002763c: mov r0, r11
0002763e: mov r0, r12
00027640: mov r0, r13
00027642: mov r0, r14
00027644: mov r0, r15

Debug x

Console x Registers Problems Debugger Console Smart Browser Memory Executables

Debug DebugOnly [Renesas GDB Hardware Debugging] [pid: 249798]

GDB Server for Renesas targets.

Version 10.4.0.v20251124-120209 [b5e5602e] (Nov 24 2025 14:52:09)

Starting server with the following options:
Raw options : /home/flynn/.eclipse/com.renesas.platform_521885881/DebugComp//RH850/e2-server-gdb -g E2 -t R7F701649 -uConnectionTimeout= 30 -umFreq= 16.00 -ulp

Connecting to RH850 Target
GDBServer endian : little
Starting target connection

Version Information:
RH850G3 Executor V4.11.00.01 [22 Feb 2024]

Finished target connection
GDB: 45879
Target connection status - OK
Target connection status - OK
Hardware breakpoint set at address 0x277d2

e2-server-gdb parameters

- Lot of parameters are passed to **e2-server-gdb**, readable through **e² studio** console:

```
e2-server-gdb -g E2 -t R7F701649 -uConnectionTimeout= 30 -umFreq= 16.00
-ulpdType= 4 -ulpdFreq= Default -setOptJtagLpd= 1 -setOptJtagJtag= 0 -uHotPlug= 0
-releaseReset= 0 -uInitialStop= 0 -w 0 -usecurityID=
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF -uneedAuthentication= 0 -uresetOnReload= 1
-ustopPeriEmu= 0 -uMultiDeviceSync= 0 -usupplyVoltageASP= 0 -umaskReset= 0
-umaskHldrq= 0 -umaskStop= 0 -umaskWait= 0 -uverifyOnWritingMemory= 0
-uWorkRamAddress= 0xfedff000 -uDebugCodeSRAM= 0 -uInitialiseRAM= 1 -uForceReset= 0
-uextflEnable= 0 -uUseSyncTrace= 0 -l -uCore= SINGLE_CORE|enabled|1|main
-uSyncMode= async -uFirstGDB= main --gdbVersion= 16.2
```

v850-elf-gdb parameters

- When v850-elf-gdb is launched, information about the memory map is transferred:

```
# Redacted for readability
set breakpoint pending on
set detach-on-fork on

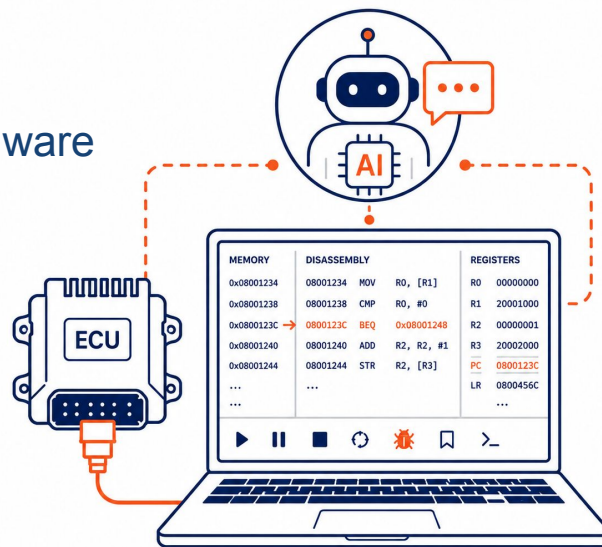
file ../firmwares/firmware.elf

set architecture v850e3v5
monitor set_target,R7F701649

monitor set_io_access_width,RW,4,
10020004,1002000c-10020014,1002001c-1002004c,10020080-10020098,100200a0-100200c8,10020128,
10020300-10020310,1002031c,10020400-10020508,10020510-10020514,10020710-10020714
# Many more monitor set_io_access_width
hbreak *0x0000
```

One tool to debug them all

- To streamline usage, we crafted one tool that:
 - Launches **e2-server-gdb**
 - Generates the required gdb start commands, including the memory map
 - Executes **v850-elf-gdb**
- It only requires
 - The **ELF** file generated from the dumped firmware
 - The RH850 microcontroller **version**
 - The **ID Code**
 - The **oscillator frequency**
 - The address of the **entry breakpoint**



Result

```
user@qb:~/RH850_debug$ ./gdb_rh850.py -t R7F701649 -f ./firmware.elf -m 0x27628 -id
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF -osc 16.0
[main_menu] Running GDB server...
[gdbserver]
[gdbserver] Connecting to RH850 Target
The target architecture is set to "v850e3v5".

Program received signal SIGTRAP, Trace/breakpoint trap.
0x00000000 in _binary_firmware_bin_start ()
Breakpoint 1, 0x00027628 in _binary_firmware_bin_start ()

(gdb) info registers PC
PC          0x27628          0x27628 <_binary_firmware_bin_start+161320>

(gdb) x/i $PC
=> 0x27628 <_binary_firmware_bin_start+161320>:    mov    r0, r1
```





Bypassing debug protection

RH850 debug protection

- RH850 debug access supports different communication modes:
 - **UART/CSI** (1-wire, 2-wire, 4-wire for flash programming)
 - LPD (1-wire, 4-wire)
 - JTAG
- It can be protected by either being:
 - **Permanently locked**
 - **Password protected** (16-byte ID Code)

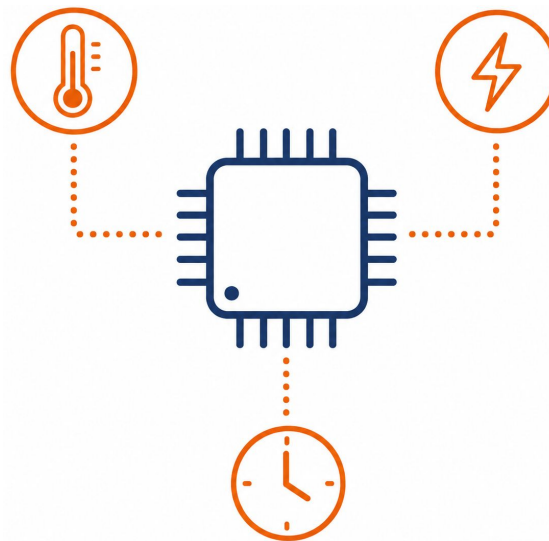
Public work on debug access recovery using fault injection:

- **2022:** [Bypassing the Renesas RH850/P1M-E read protection using fault injection](#)
- **2024:** [Unlocking Renesas RH850/F1L Security with Voltage Glitching](#)
- **2026:** [Bypassing debug password protection on the RH850 family using fault injection](#)

Fault injection - hardware constraints

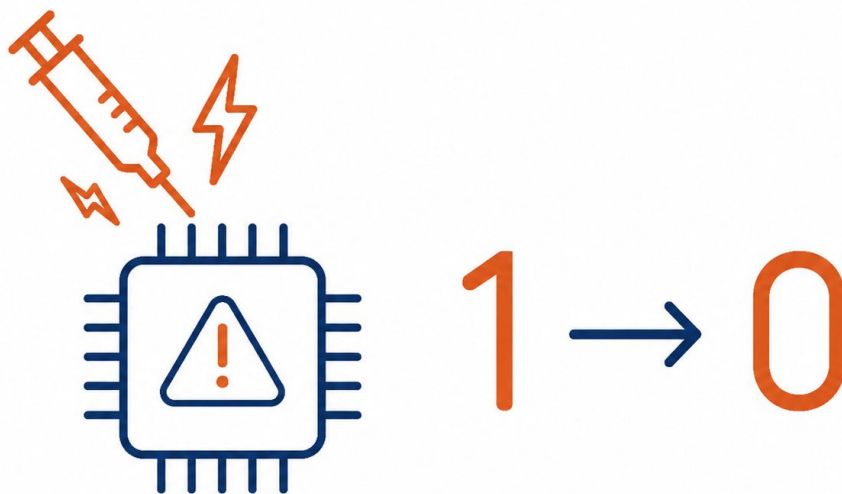
Electronic systems are made up of various components (memory chips, microcontrollers, ...) that must operate within an **operating range**:

- Voltage
- Clock frequency
- Temperature
- ...



Fault injection - effect

A fault injection attack aims to influence **these operating parameters** in order to reach a state of **instability**, so as to generate **corruption** in the data stored in memory, transiting between components, ...



Fault injection - attack types

Several fault injection attack types exists:

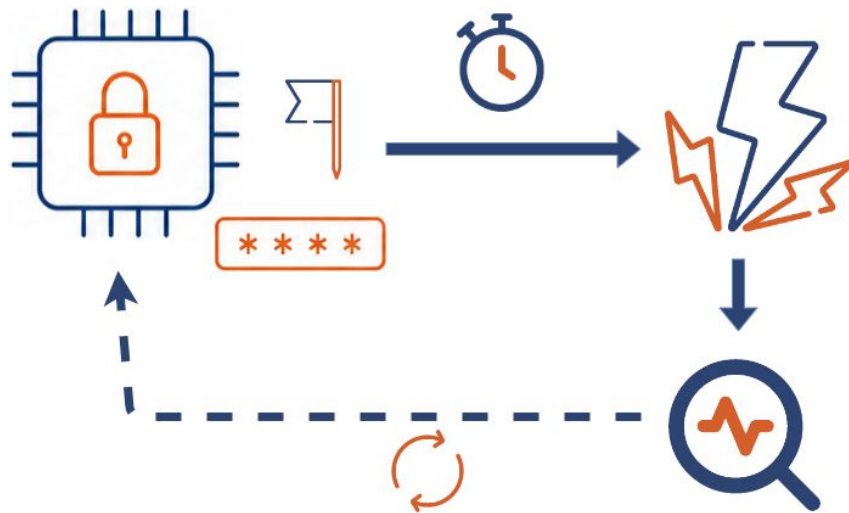
- Clock glitching
- Voltage glitching
- Electromagnetic pulse
- Laser
- ...



How an attack works

To carry out an attack on a specific function of a system, the following parameters must be identified:

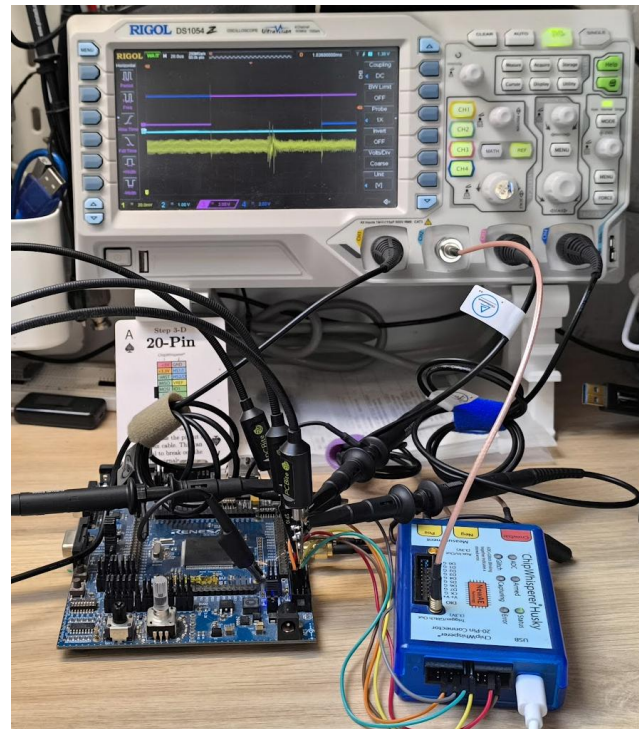
- Trigger
- Delay
- Injection parameters



The reliability and repeatability of an attack rely on identifying a **deterministic trigger**

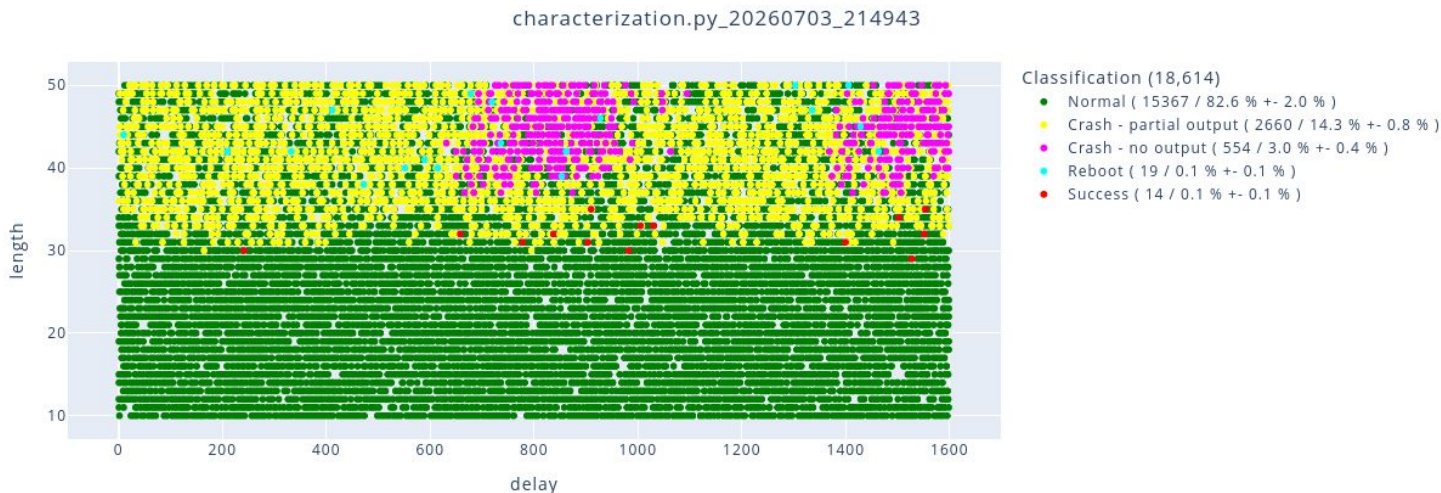
Fault injection setup

- **Attacked feature:** 16-byte password over 2-wire UART
- **Target:** RH850/F1KM
- **Fault injection type:** voltage glitching
- **Tool:** Chipwhisperer Husky



Target preparation & characterization

- Using voltage glitching, capacitors on voltage internal regulator pin were removed
- A **test script** was crafted to determine if the target is vulnerable to fault injection and get base parameters



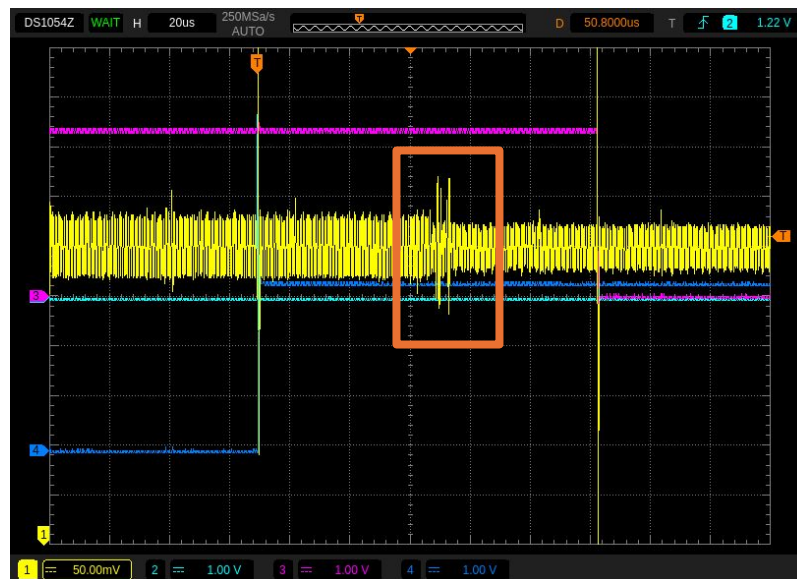
Identifying attack trigger and attack window

- 2-wire programming access use **UART protocol** to send the ID Code
- The **last byte** of the ID Code packet is used as a **trigger** (blue)
- Monitoring the UART response of the RH850 give the **attack window** (yellow)
- There is a **jitter** in the attack window:
-> **the trigger is not deterministic**



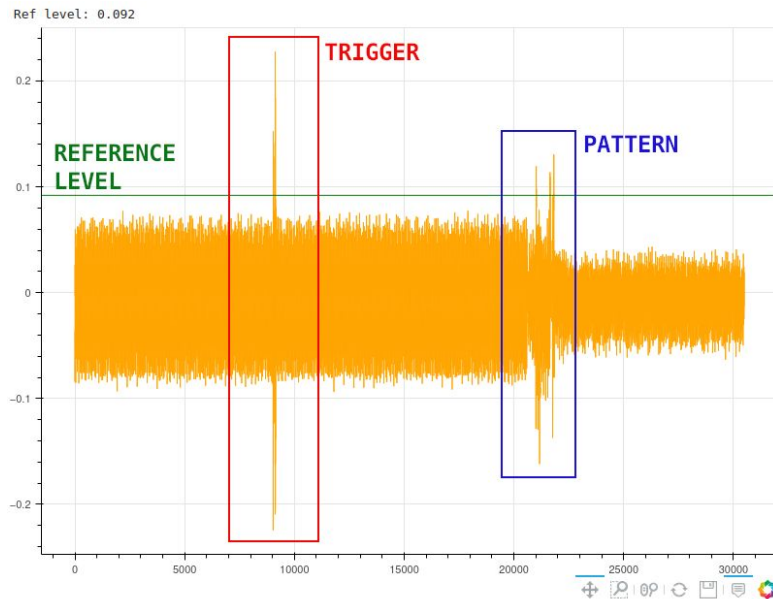
Using side-channel power analysis

By monitoring the **power consumption** of the RH850, we noticed a **specific pattern** between the transmission of the ID Code and the response of the microcontroller



Using the alternative trigger

To enhance our attack and bypass the **jitter** issue, we define a **reference level** on the power consumption that will be used as a trigger, once the password is sent



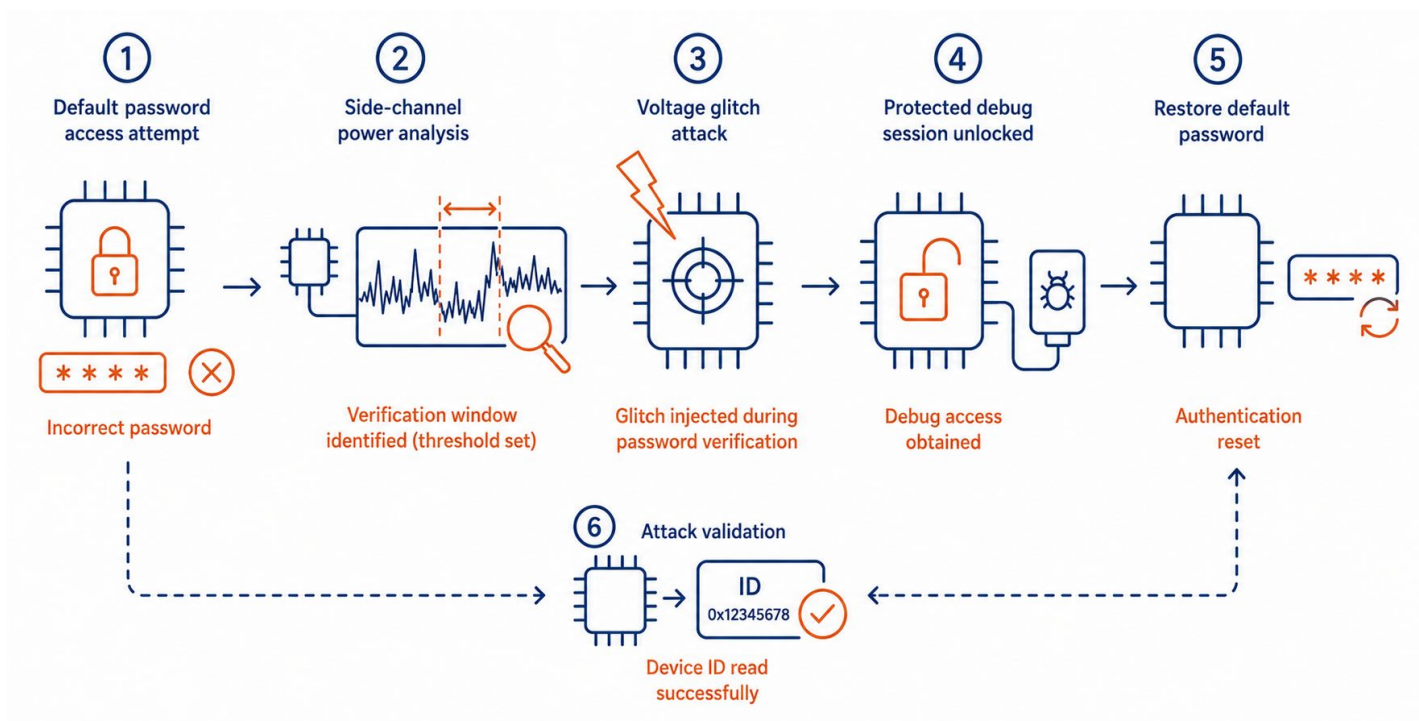
Results

```
user@qb:~/RH850_glitch$ ./run_RH850_glitch.py
```

```
Delay: 537 Width: 18, Crash 2, Reset: 25: 0% | | 88/16800 [01:10 < 35:52:39, 1.33it/s]
```

```
[SUCCESS] IDCODE response: b'\x81\x00\x01\x3a\xc5\x03'  
Reading option 0x3A: R7F701649
```

Full attack flow



Questions?



RH850 ARCHITECTURE
INSTRUCTION SET REFERENCE
DEBUGGING PRINCIPLES
HARDWARE MASTERY



Quarkslab



10, Boulevard Haussmann
75009 PARIS

www.quarkslab.com